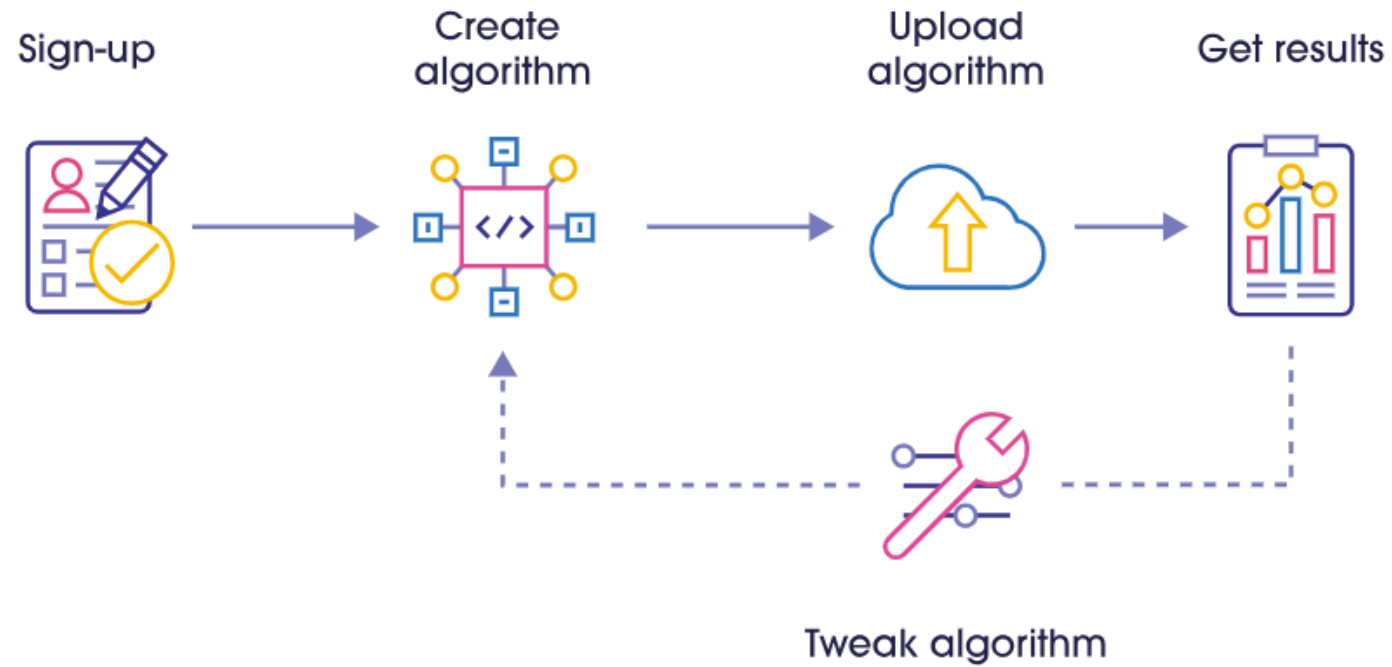


WP module Learning Optimization DPC – W1

Prof. Dr. Thomas Winter, Dr. Alwin Haensel

DPC - dynamic-pricing-competition.com

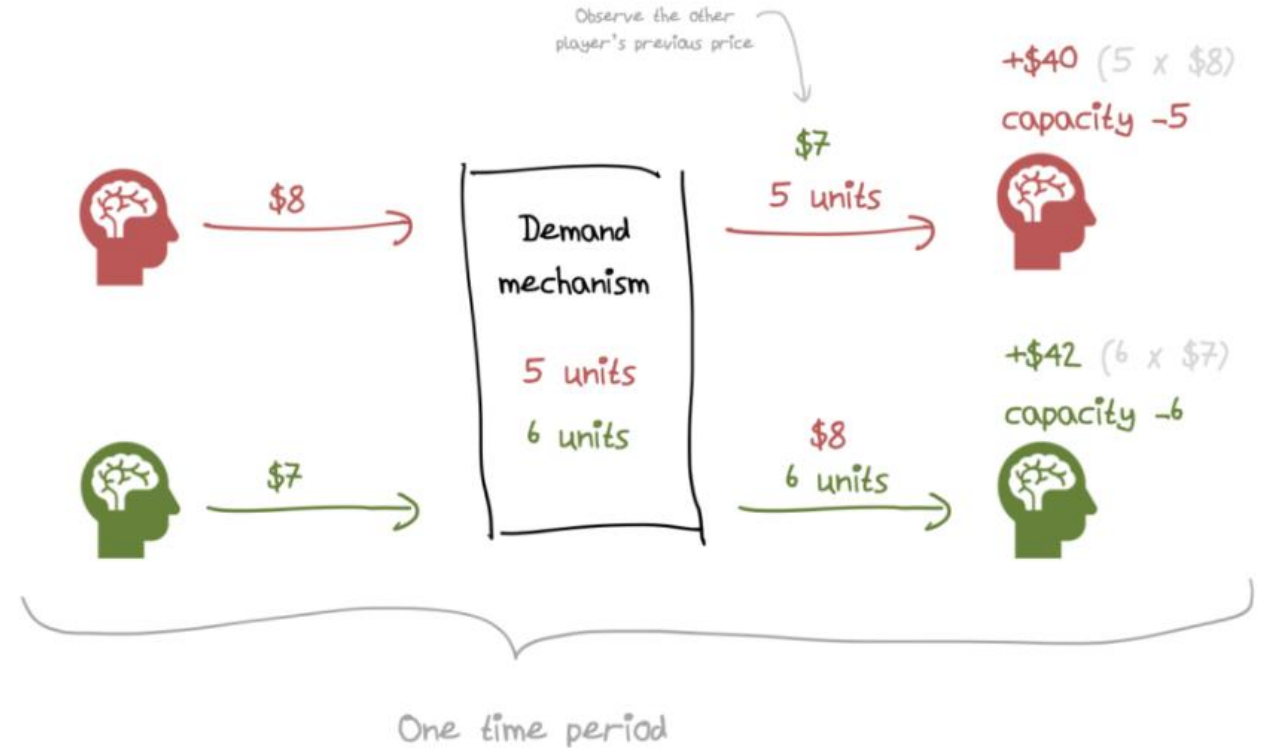
The DPC is a simulation environment for pricing algorithms



DUOPOLY

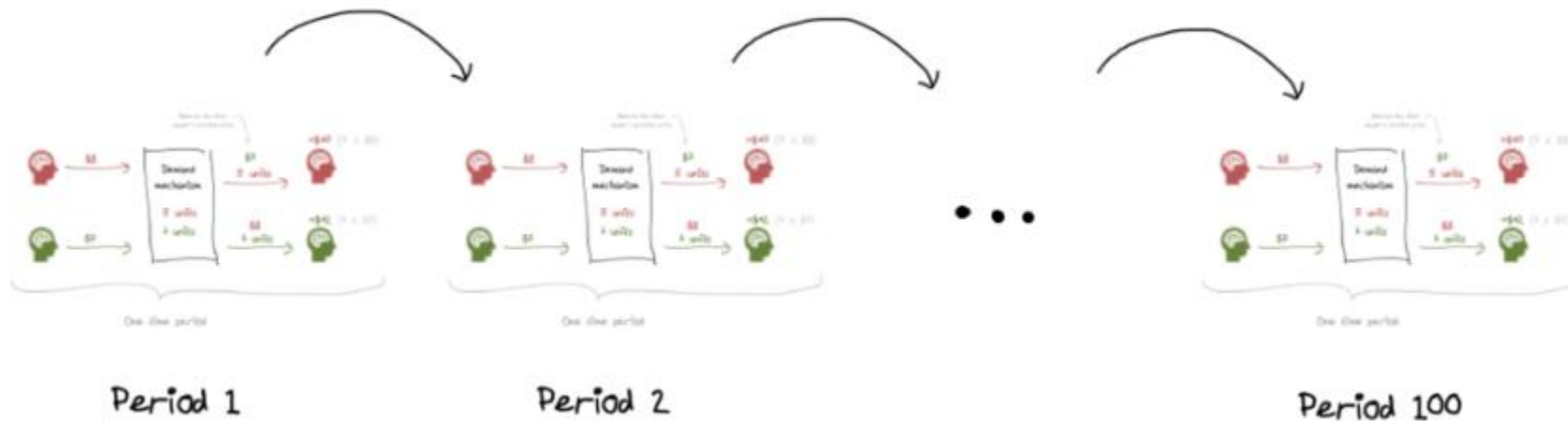
Airline Example.

- 2 Player
- Each gives a price
- The demand model computes the utility for each price for each customer
 - high price $\rightarrow$ low utility/ attractiveness, *but high revenue for us*
 - low price $\rightarrow$ high utility/ attractiveness, *but low revenue for us*



DUOPOLY

- 1 *simulation* consist of 100 *selling seasons*, during one simulation
 - the *demand model is the same for all selling seasons*
 - you compete against the same competitor in all selling seasons
- 1 *selling season* consists of 100 *selling periods* (=days)
- In each selling season you have a *capacity* of 80 units (= seats in a plane) to sell
- If one player sells all 80 units of capacity before selling period 100, the other sells alone



Most Basic Algo: duopoly.py

- File name `duopoly.py`
- Pricing function `p(.)`
 - Returns: *Tupel (price, object)*

```
from typing import Any, Optional, Tuple, Union
import numpy as np

def p(
    current_selling_season: int,
    selling_period_in_current_season: int,
    prices_historical_in_current_season: Union[np.ndarray, None],
    demand_historical_in_current_season: Union[np.ndarray, None],
    competitor_has_capacity_current_period_in_current_season: bool,
    information_dump=Optional[Any],
) -> Tuple[float, Any]:

    price = np.random.uniform(30, 80)

    return price, None
```

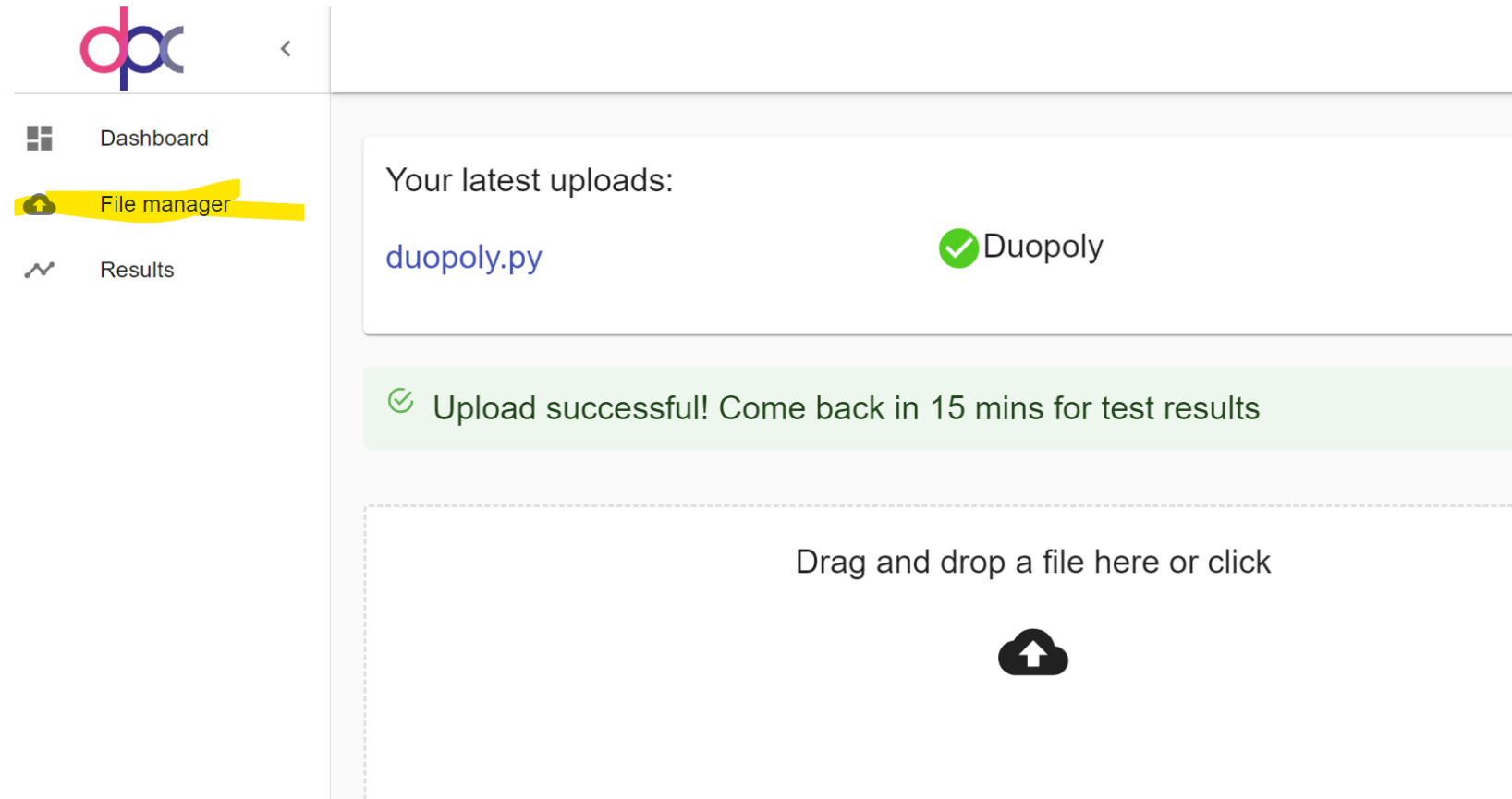
Input Arguments / Information available

- ***current_selling_season*** : 1...100
- ***selling_period_in_current_season*** : 1..100
- ***prices_historical_in_current_season*** : 2-dim array of historical prices. The rows = competitors, columns = periods
- ***demand_historical_in_current_season*** : 1-dim array of historical (own) demand.
- ***competitor_has_capacity_current_period_in_current_season*** : `False` if competitor is out of stock
- ***information_dump*** : Any what you like to pass to yourself

```
def p(  
    current_selling_season: int,  
    selling_period_in_current_season: int,  
    prices_historical_in_current_season: Union[np.ndarray, None],  
    demand_historical_in_current_season: Union[np.ndarray, None],  
    competitor_has_capacity_current_period_in_current_season: bool,  
    information_dump=Optional[Any],  
) -> Tuple[float, Any]:
```

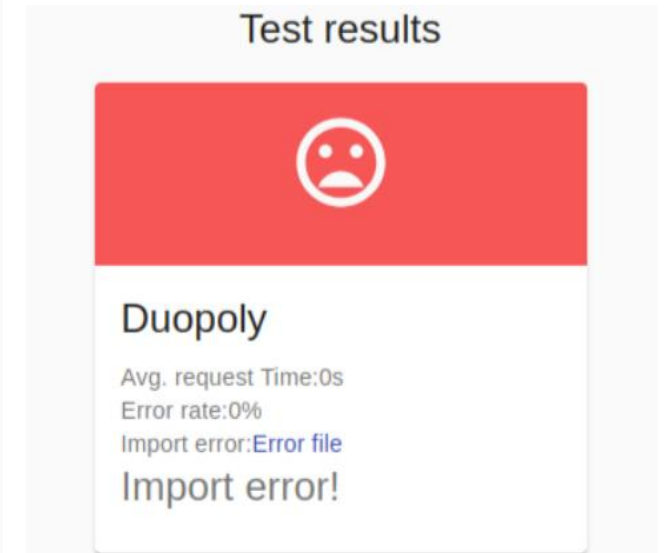
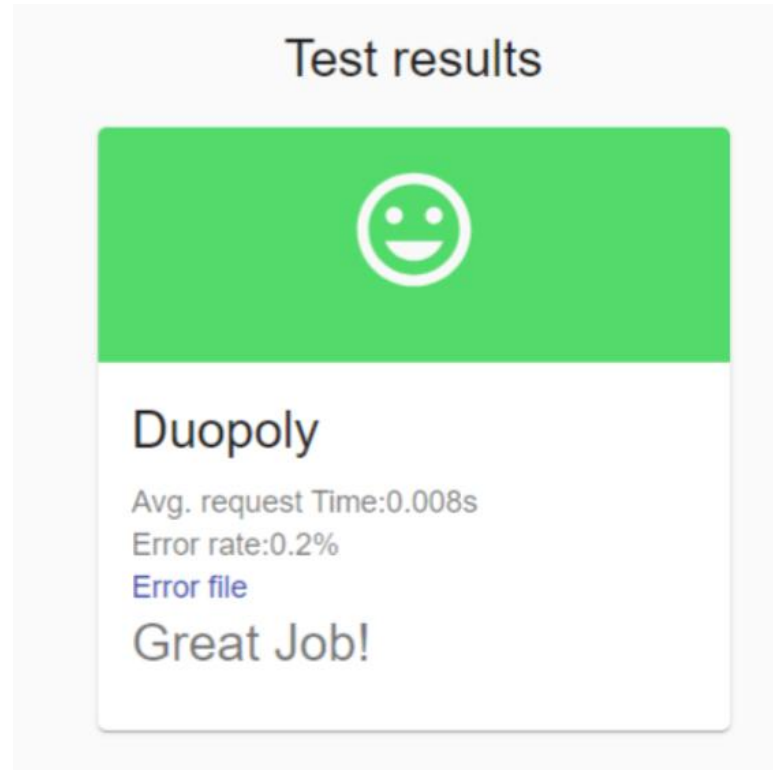
Algorithmus Test in the UI

- *File upload*
- *One file must be named **duopoly.py***
- *Wait 10-15min*

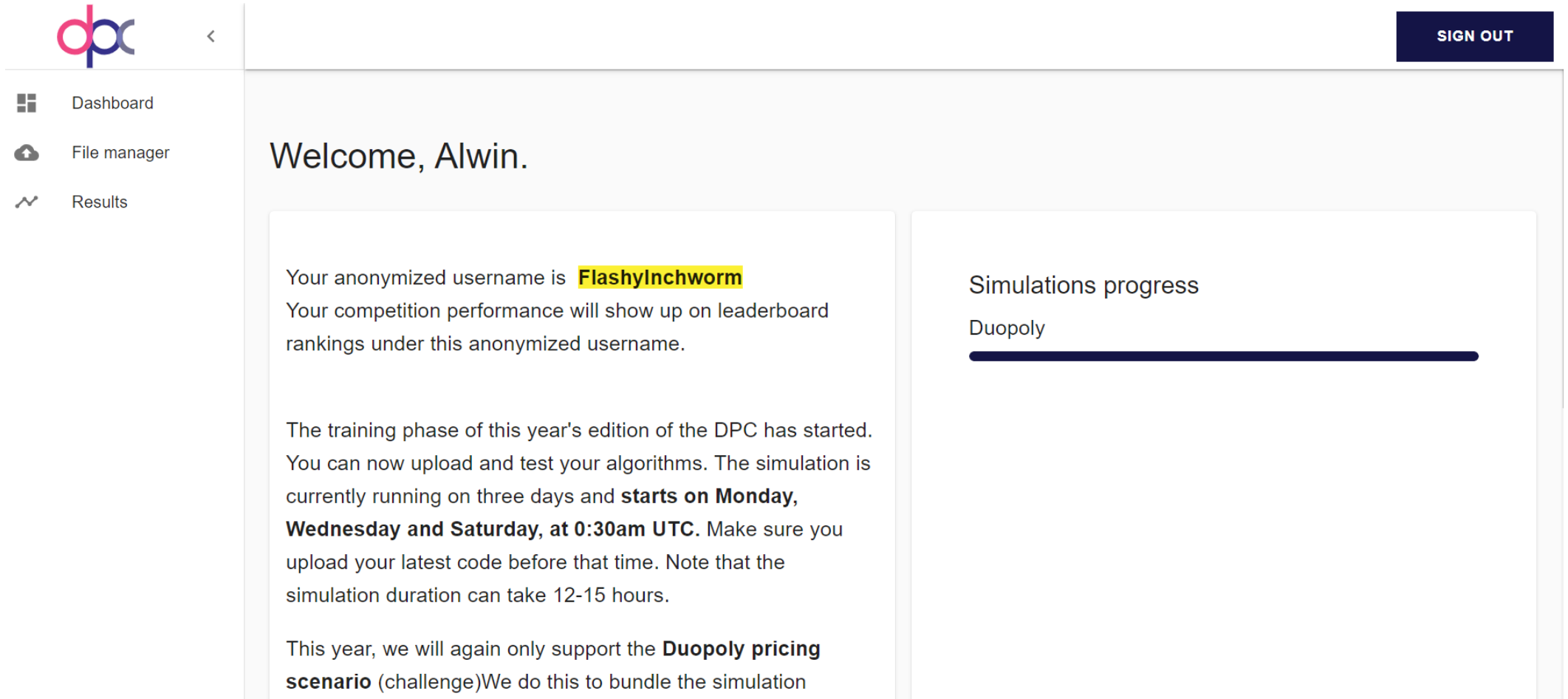


Algorithmus Test in the UI

- *File upload*
- *One file must be named **duopoly.py***
- *Wait 10-15min*



DPC Tour



The screenshot shows the DPC Tour dashboard. On the left is a sidebar with the DPC logo and navigation links: Dashboard, File manager, and Results. The main content area is titled 'Welcome, Alwin.' and contains three sections. The first section displays the user's anonymized username 'FlashyInchworm' and states that their competition performance will be shown on the leaderboard. The second section announces the start of the training phase, noting that the simulation is running for three days and starting on Monday, Wednesday, and Saturday at 0:30am UTC. The third section mentions that only the Duopoly pricing scenario is supported this year. On the right, there is a 'Simulations progress' section with a progress bar for 'Duopoly'.

 <

SIGN OUT

 Dashboard

 File manager

 Results

Welcome, Alwin.

Your anonymized username is **FlashyInchworm**
Your competition performance will show up on leaderboard rankings under this anonymized username.

The training phase of this year's edition of the DPC has started. You can now upload and test your algorithms. The simulation is currently running on three days and **starts on Monday, Wednesday and Saturday, at 0:30am UTC**. Make sure you upload your latest code before that time. Note that the simulation duration can take 12-15 hours.

This year, we will again only support the **Duopoly pricing scenario** (challenge) We do this to bundle the simulation

Simulations progress

Duopoly

<https://dynamic-pricing-competition.com/getting-started/>

[Home](#)[Challenge](#)[Getting Started](#)[Timeline](#)[Winners](#)[LOGIN/REGISTER](#)

Algorithm requirements

- Your code will be executed in a docker container with `Python 3.9` and `Anaconda distribution 2022.05`. We have also added: `tensorflow==2.13.0`, `tf-agents==0.14.0`, `tornado==6.3.3`, `torch==2.0.1`, `xgboost==1.6.2`, `botorch==0.8.5`. If you need further packages, please contact us.
- The container in which your code runs has `1 CPU and 512 MB` of memory and your code is not allowed to use the local file system to save your own log files or anything like this. Further, there is no need for printing out things in your functions, we are not logging your output.
- When uploading your algorithms on the platform, please name the pricing algorithms `"duopoly.py"` (1-airline case). This is essential for the competition setup on AWS.
- The competition setup expects that each of the main scripts contains a `function p(.)`, which we will call, the function is expected to `return the price and the information dump`.
- The `information dump` allows you to pass yourself any kind of information from one time period to the next. But note, that the information dump is reset to `None` at the beginning of every competition. So you can use the information dump only during the competition to pass back information to yourself in the next time period, but not across simulations. For this you can use the `'duopoly_feedback.data'` file, see explanation below.
- You can use supporting python scripts by simply importing them in your scripts from the current working directory. All your files will be placed in the same directory.
- Your algorithms need to return a price response on average within 0.2 seconds per time period/iteration and the maximum time for a single price response is 5 seconds. Further, the algorithm needs to be error free in 99% of the price requests.
- In case we do not receive a valid price response (`>0`) from your function, we will use your old price from the previous time period. In case your function is not working on the first period in the selling season, we will draw a random price and use that.

Let's get the first Algo going

[Home](#)[Challenge](#)[Getting Started](#)[Timeline](#)[Winners](#)[LOGIN/REGISTER](#)

Challenge and example code

1

A simple example algorithm code with the explanation of the main functionalities can be found [here](#).

General setup in the challenge:

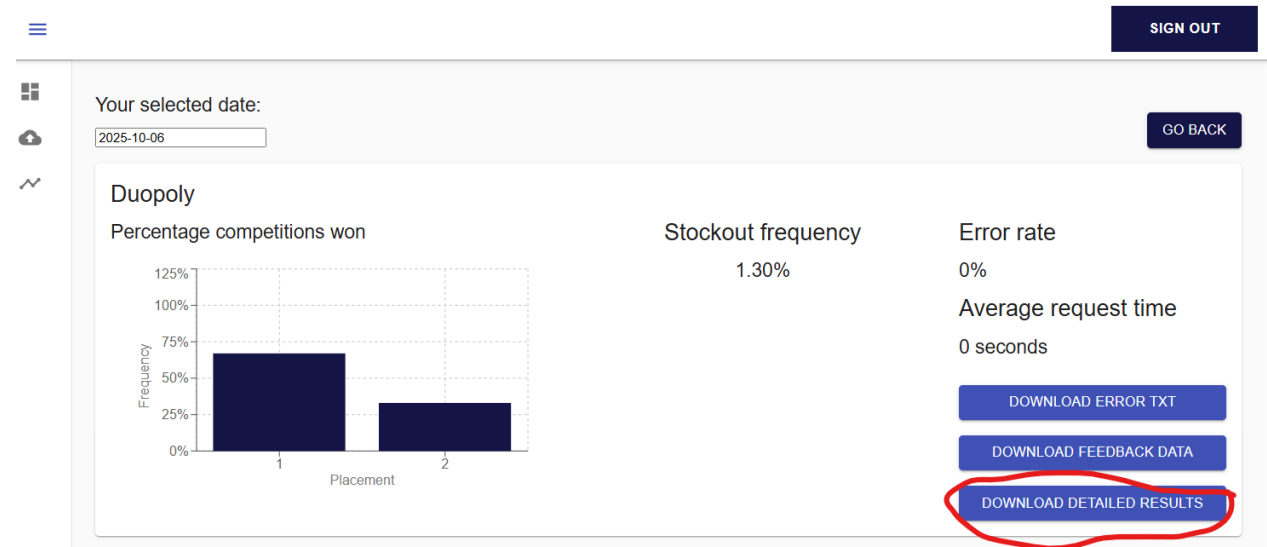
Each time period, each player posts a price, this is given as input into the unknown demand mechanism. Each player in the competition receives then their own sales realization and competitors' prices. The demand mechanism that generates the sales based on the prices is unknown to all competitors.

Players are randomly selected to play against each other in small competition matches. Each match consists of multiple simulations, and the player with the overall highest generated revenue overall simulations is winning the match.

Duopoly: the airline challenge

How to read the results

- Winner is the player with the highest total revenue over all simulations
- You can download detailed results in a CSV



Ranking

Search

Overall Ranking	Name	Total Revenue	Avg Revenue per Simulation	Avg Ranking per Simulation	Min Set Price	Max Set Price	Mean Price	Autocorrelation Price	Stockout Frequency
1	FearlessFinch	\$2,304,939	\$329,277	1.29	\$0.10	\$99.90	\$43.77	0.676	0.467
2	WhimsicalMarten	\$2,050,995	\$292,999	1.14	\$0.10	\$99.00	\$37.82	0.644	0.55
3	ElectricButterfly	\$2,010,124	\$251,266	1	\$0.10	\$99.00	\$31.77	0.701	0.394
4	ImposingCrocodile	\$1,989,914	\$284,273	1.43	\$40.00	\$70.00	\$54.99	-0.019	0.233

Results CSV -> very helpful

- *competition_id* = ID of this competition run (100 selling seasons)
- *selling_season* = (int) 1..100
- *selling_period* = (int) 1..100
- *competitor_id* = name of competitor
- *price_competitor* = price of competitor
- *price* = your price
- *demand* = realized demand for you (-> your revenue is $\text{price} * \text{demand}$)
- *competitor_has_capacity* = True / False
- *calculation_duration* = response time of your algo in seconds

	A	B	C	D	E	F	G	H	I	J
1	competition_id	selling_season	selling_period	competitor_id	price_competitor	price	demand	competitor_has_capacity	calculation_duration	errors
2	32RpJR	1	1	FunkyGoshawk	75	54.5	2	TRUE	0.0120	
3	32RpJR	1	2	FunkyGoshawk	50	54.5	0	TRUE	0.0110	
4	32RpJR	1	3	FunkyGoshawk	25	54.5	0	TRUE	0.0100	
5	32RpJR	1	4	FunkyGoshawk	22.5	54.5	0	TRUE	0.0100	
6	32RpJR	1	5	FunkyGoshawk	20.2	54.5	0	TRUE	0.0060	
7	32RpJR	1	6	FunkyGoshawk	18.2	45.4	0	TRUE	0.0120	
8	32RpJR	1	7	FunkyGoshawk	97.4	45.4	0	TRUE	0.0130	
9	32RpJR	1	8	FunkyGoshawk	0.5	45.4	0	TRUE	0.0070	
0	32RpJR	1	9	FunkyGoshawk	35.2	45.4	0	TRUE	0.0130	
1	32RpJR	1	10	FunkyGoshawk	27.9	45.4	0	TRUE	0.0100	
2	32RpJR	1	11	FunkyGoshawk	20.8	31.8	0	TRUE	0.0070	
3	32RpJR	1	12	FunkyGoshawk	12.3	31.8	0	TRUE	0.0110	
4	32RpJR	1	13	FunkyGoshawk	14	31.8	0	TRUE	0.0090	
5	32RpJR	1	14	FunkyGoshawk	3.9	31.8	0	TRUE	0.0080	

	A	B	C	D	E	F	G	H	I	J	K	L	M	
1														
2														
3	Sum of demand	Column Labels												
4	Row Labels		1	2	3	4	5	6	7	8	9	10	11	12
5	AttentiveOriole													
6	333oHT		44	64	51	40	62	56	61	62	54	56	61	40
7	VH9Whp		39	63	45	41	61	51	57	61	53	55	61	39
8	FunkyGoshawk													
9	32RpJR		37	55	42	34	57	43	52	58	45	42	56	30
10	SqEST2		33	48	39	29	53	40	48	46	43	28	43	40
11	StoicCentipede													
12	dg7a7e		80	58	72	53	56	60	48	67	52	55	64	60
13	SBqznu		80	55	66	51	49	59	53	70	64	67	71	58
14	(blank)													
15	(blank)													
16	Grand Total		313	343	315	248	338	309	319	364	311	303	356	267
17														
18														
19														
20														

PivotTable Fields

Choose fields to add to report:

☒ competition_id

☒ selling_season

☐ selling_period

☒ competitor_id

☐ price_competitor

☐ price

☒ demand

☐ competitor_has_capacity

☐ calculation_duration

More Tables...

Drag fields between areas below:

Filters

Columns

selling_season

Rows

competitor_id

competition id

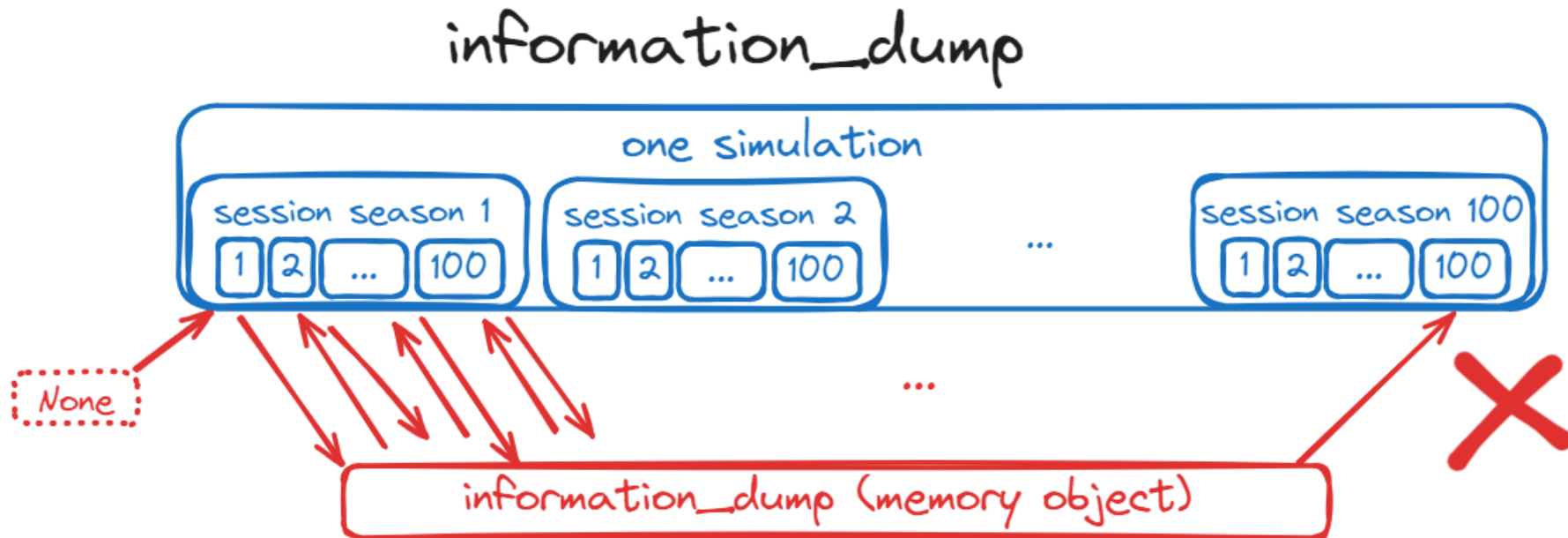
Values

Sum of demand

☐ Defer Layout Update

Information dump

= in-memory object to pass to yourself information within the selling season,
not across selling seasons



duopoly_feedback.data File

= per night run a new local file in container, which is the next day available in the UI for download, **it can cover all simulations of the nightly run**

duopoly_feedback.data

